

24 Sep 2026

Topics to learn : Repositories, commits, branches, pull requests, merge conflicts.

Watch: Traversy Media - Git & GitHub for Beginners.

Done : Create a repository; use a feature branch, PR, review, and resolve one merge conflict.

GIT

Git is a popular version control system.

It was created by Linus Torvalds in 2005, and has been maintained by Junio Hamano since then.

- Tracking code changes
- Tracking who made changes
- Coding collaboration

-
- **Repository:** A folder where Git tracks your project and its history.
 - **Clone:** Make a copy of a remote repository on your computer.
 - **Stage:** Tell Git which changes you want to save next.
 - **Commit:** Save a snapshot of your staged changes.
 - **Branch:** Work on different versions or features at the same time.
 - **Merge:** Combine changes from different branches.
 - **Pull:** Get the latest changes from a remote repository.
 - **Push:** Send your changes to a remote repository.
-

Git and Github

Git is the tool to track the changes over time.

Github is the website where we host all of our git repository.

Version control : The management of changes to documents, computer programs, large websites and other collections of information.

Directory : Folder

Repository : Project or the folder contains multiple directories

Git Commands



README.md : markdown

It has some features like one

- # tag for main header
- Description

.git is hidden folder inside the git repo

.git stores all of the files that save your commits, or your code changes over time all of the change recorded in the history of this repo

Git status command shows the files that were updated or created or deleted, but haven't been saved in commit yet.

Connect local machine to our account

By using **SSH** keys we can add local to remote platform

While adding the local repo to the remote repo we will see this error :

```
git push origin master
```

```
fatal: 'origin' does not appear to be a git repository
```

```
fatal: Could not read from remote repository.
```

```
Please make sure you have the correct access rights  
and the repository exists.
```

So at the place of this error we have to add own **origin**

The command `git remote add origin <link>` is used to connect your local project directory to a remote server (like [GitHub](#), [GitLab](#), or [Bitbucket](#)).

Think of it as creating a bookmark or shorthand shortcut for a very long web address. [1, 2]

Here is exactly what each part of the command does: [1]

- `git remote add`: Tells Git that you want to register a brand new link to an external repository. [1, 2]
- `origin`: This is simply a nickname (alias) you choose for that link. By default, developers use "origin" out of convention, but you could technically name it "github", "backup", or anything you want. [1, 2]
- `<link>`: The actual URL of your hosted repository (e.g., `https://github.com`). [1]

Why do we need it?

Without this command, every single time you want to upload or download code, you would have to type out the entire web URL. [1, 2]

By saving it under the nickname **origin**, you can use quick commands like:

- `git push origin main` (to upload your code)
- `git pull origin main` (to download the latest updates) [1, 2]

Git push -u origin master

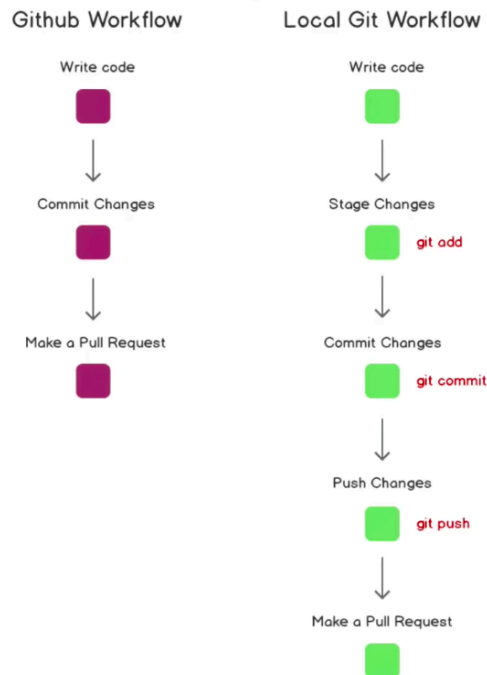
Setting upstream tells git to remember this connection of local to remote branch

The command `git push -u origin master` means **you are uploading your local code to the remote server for the very first time and telling Git to remember this connection.**

Here is the exact breakdown of what each part does:

- `git push`: The action command that uploads your local project files and history to the internet.
- `-u (Upstream)`: This is the magic flag. It tells Git to **link** your current local branch to the remote branch. You only need to use `-u` the **very first time** you push. After that, Git will remember the connection, and you can just type a shortcut: `git push`.
- `origin`: The nickname for your remote repository URL (the one you set up using `git remote add`).
- `master`: The name of the specific branch you are uploading. (*Note: Many modern platforms like GitHub now use `main` instead of `master` as the default branch name*).

Local Git Workflow



GIT BRANCHING

How to merge two branches

To complete a git merge, you need a sequence of commands. **There is no single command that safely completes a merge from start to finish.**

Here is the exact step-by-step sequence to merge a feature branch (e.g., `feature-login`) into your main branch:

1. Switch to the receiving branch

First, you must move into the branch that you want to merge *into* (usually `main` or `master`).

```
git checkout main
```

2. Pull the latest updates

Always download the newest changes from the cloud to prevent conflicts.

```
git pull origin main
```

3. Run the merge command

Now, pull the changes from your feature branch into `main`.

```
git merge feature-login
```

What happens next? (Two Possible Paths)

Depending on your code, one of two things will happen after step 3:

Path A: The Merge is Successful (No Conflicts)

If there are no overlapping code changes, Git completes the merge automatically. A text editor might pop up asking for a merge message. Just save, close it, and upload the final result:

```
git push origin main
```

Path B: You get a "Merge Conflict"

If two branches modified the exact same line of code, Git stops and prints:

```
CONFLICT (content): Merge conflict in <file-name>.
```

To complete the merge in this case, follow these steps:

1. Open the conflicted file in your code editor (like VS Code).
2. Look for the conflict markers (`<<<<<<`, `=====`, `>>>>>>`) and choose which code to keep.
3. Save the file.
4. Tell Git you resolved it by running:

```
git add <file-name>
```

5.

6. Complete the merge process:

```
git commit -m "Resolved merge conflict"
```

7.

8. Push it online:

```
git push origin main
```

9.

Undoing in Git

We can undo our stages or commits in git

If we mistakenly add something that we doesn't want then we can undo our commands

Undo a file :

Git reset <name of the file>

If we want to undo our **add** . file that we have added

Then we can use this command.

Undo a commit :

Git reset HEAD~1

It will undo last commit we did, also **head** is pointing to the last most commit

Check all the commits :

To check all the commits we do **git log**

Git fork

No, there is no standard `git fork` command in Git. [1]

Just like Pull Requests, a **Fork** is a feature belonging to the **GitHub web interface** (and similar platforms like GitLab or Bitbucket), not the core Git software installed on your computer.

When you "Fork" a repository, you are making a **personal copy** of someone else's entire project onto your own GitHub account so you can experiment with it without modifying their original code.

Advanced Git commands : that developers use to fix mistakes, clean up history, and manage complex code bases.

They are grouped by what they help you achieve:

1. Cleaning Up & Organizing History

When you want your commit history to look neat and professional before sharing it.

- **git commit --amend**
 - **What it does:** Changes the message or adds files to your very last commit without creating a brand new one.
 - **Use case:** You just committed your code, but realized you made a typo in the commit message or forgot to include one file.
- **git rebase -i HEAD~n (Interactive Rebase)**
 - **What it does:** Opens an interactive menu for your last `n` commits (e.g., `HEAD~3` for the last 3 commits). It allows you to squash (combine) multiple messy commits into one, reorder them, or delete them entirely.
 - **Use case:** Cleaning up your branch's history before opening a Pull Request.

2. Moving Changes Between Branches Safely

When standard merging is too messy or broad.

- **git cherry-pick <commit-id>**
 - **What it does:** Grabs a single, specific commit from another branch and copies it onto your current branch, leaving the rest of that branch behind.
 - **Use case:** A coworker wrote a specific bug fix on their branch, and you need *only* that fix right now without merging their entire unfinished feature.
- **git rebase <branch-name>**
 - **What it does:** Takes all your new commits on your current branch, lifts them up, and places them neatly on top of the latest commit of the target branch. It creates a completely linear history instead of a messy merge bubble.
 - **Use case:** Keeping your feature branch up-to-date with `master` without creating ugly "Merge branch master into..." commits.

3. Finding Bugs and Lost Code

When something broke and you need to investigate the past.

- **git reflog**
 - *What it does:* Shows a detailed, unalterable log of every single move your local Git head made (every checkout, reset, commit, or merge).
 - *Use case:* The ultimate undo safety net. If you accidentally did a **git reset --hard** and lost your work, **git reflog** helps you find the hidden commit ID to bring it back.
- **git bisect**
 - *What it does:* Uses a binary search algorithm to look through your project history. It asks you to mark a past commit where the app worked ("good") and the current commit where it's broken ("bad"). Git will automatically hop between commits and ask you to test them until it isolates the exact commit that introduced the bug.
 - *Use case:* Finding a bug that was introduced weeks ago when you don't know which file caused it.

4. Advanced Inspecting

- **git log --oneline --graph --decorate --all**
 - *What it does:* Draws a visual text-based graph map of all your branches and commits directly inside your terminal.

Would you like to deep dive into one of these commands? If you want, tell me:

- A specific problem you are trying to solve right now (e.g., "I want to combine 3 messy commits into one" or "I lost a commit").
- If you want a step-by-step breakdown of how **git rebase** differs from **git merge**.

Let me know which scenario you want to explore next!

25 Sep 2026

Jira

Three main building blocks of JIRA are :

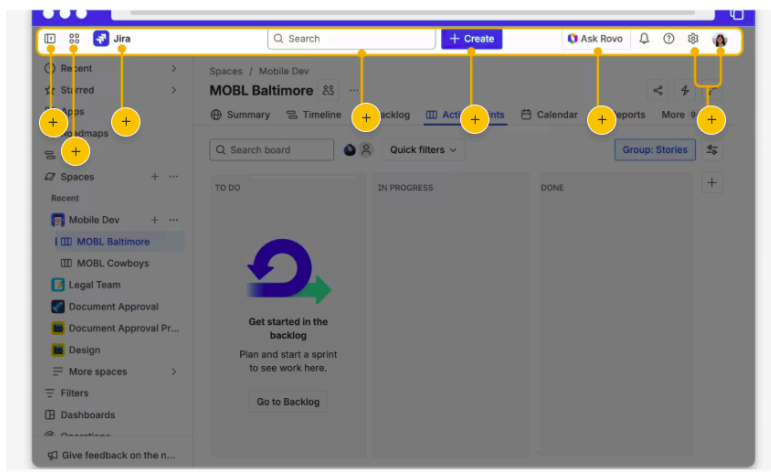
- Space
- Work items
- Views : List view and Board view

Work Items : In Jira, individual pieces of work are **work items**. You'll update work items to show work progress or add relevant information, like notes from a meeting or a question for a teammate.

Space Keys : "Space keys are short versions of the space name that identify the work items in that space."

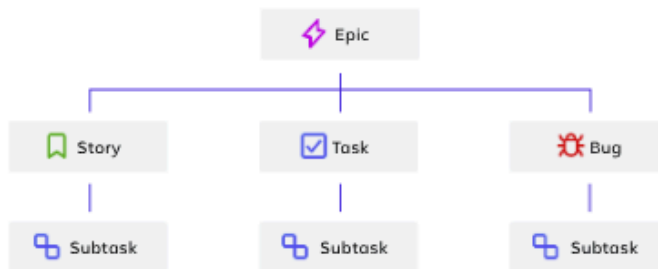
Board : A **board** is a visualization of the work in a space. Boards have columns with work items in them. The columns represent the **statuses** that your work, represented by work items, moves through. Your entire board represents your team's **workflow**, the path of statuses work items moves through.

Jira Navigation :



- **Sidebar toggle:** Show or hide the sidebar (Tip: Press [to display or collapse the sidebar).
- **App switcher:** Allows you to switch to other Atlassian sites and apps, like Confluence or Bitbucket.
- **Your work:** The app you are currently using, in this case, Jira.
- **Search:** Find work items, spaces, boards, queues, filters, and people across all apps on your site.
- **Create:** Enables you to quickly create new work items.
- **Ask RoVo:** Open RoVo chat for assistance and support.
- **Settings and profile:** Customize your personal settings, set up your profile, and control email notifications.

Jira Work Types :



Work type batata hai ki ye work item kis category ka hai aur uska scope/size kya hai.

Jira = team ke work ko manage aur track karne ka platform.

Rovo = Atlassian ka AI layer jo Jira aur connected company data ko samajhkar search, summarize, create aur automate karne mein help karta hai.

Main features:

- 🔍 **Search** → Jira, Confluence aur connected sources mein information find karna
- 💬 **Chat** → company/project context ke basis par questions poochna
- 🤖 **Agents** → specialized AI workers jo configured tasks perform kar sakte hain
- ⚙️ **AI Automation** → natural language se automation create/help
- 🧠 **Knowledge** → company terminology/context samajhne mein help
- 📊 **Deep Research** → multiple sources se information analyze karke report banana

WORK ITEMS Updation :

Dependencies of Work Items :

Let's take four examples:

Relates to

Login UI ↔ Login API

They're connected, but no dependency is necessarily implied.

Depends on

Checkout



depends on



Payment API

Checkout needs Payment API.

Blocks

Payment API



Checkout Testing

Payment API is preventing Checkout Testing from proceeding.

Clones

Original Bug



Clone

One work item is a copy of another.

DashBoard in JIRA

Dashboard in JIRA works as a marketer

DASHBOARD SHORTCUTS

Keyboard shortcuts

Global shortcuts

Quick search

/

Close drawer

Esc

Create work item

C

Keyboard shortcuts

?

Find work items

G then I

Browse to a space

G then P

Toggle sidebar

I

Open help panel

H

Open command palette

Cmd + K

Space navigation

Go to Summary

1

Go to Timeline

2

Go to Backlog

3

Go to Active sprints

4

Go to Calendar

5

Go to Reports

6

Go to List

7

Go to Forms

8

Go to Goals

9

Navigating issues

View selected issue

O

Next issue

J

Focus search field

F

Search for issues

U

Disable keyboard shortcuts

Close

Conference app connector manual